Πολυτεχνική Σχολή

Τμήμα Μηχανικών Η/Υ & Πληροφορικής

Δομές Δεδομένων

Project Δευτέρου Συνόλου Ασκήσεων 2025-2026

Ανάλυση Αλγορίθμων και Δυναμικών Δομών Δεδομένων σε Σύνολο Εμπορικών Δεδομένων COVID-19

Γλώσσα Υλοποίησης: C

Λιόντου Δέσποινα
A.M: 1108254
Έτος εισαγωγής: 2024
up1108254@ac.upatras.gr

Καρπέτας Απόστολος
A.M: 1115507
Έτος εισαγωγής: 2024
up1115507@ac.upatras.gr

Διδάσκοντες:
Χρήστος Μακρής,
Σπυρίδων Σιούτας,

Πάτρα, 2026

1. Εισαγωγή

- **Παρουσίαση Dataset**: Αναφορά στο αρχείο "Effects of COVID-19 on trade" με τις 111.438 εγγραφές.
- **Πεδία Ανάλυσης**: Date, Value, Cumulative.

2. PART I: Αλγόριθμοι Ταξινόμησης και Αναζήτησης

- **Ταξινόμηση (Sorting)**: Σύγκριση Counting vs Merge Sort (Date/Cumulative) και Heap vs Quick Sort (Value).
- **Αναζήτηση (Searching)**: Σύγκριση Δυαδικής (Binary) και Παρεμβολής (Interpolation).
- **BIS & BIS***: Ανάλυση της Δυικής Αναζήτησης Παρεμβολής και της βελτιωμένης παραλλαγής της για τη χειρότερη περίπτωση.

3. PART II: Δέντρα BST και Κατακερματισμός

- **Δυαδικά Δέντρα (BST/AVL)**: Υλοποίηση αναζήτησης, τροποποίησης και διαγραφής βάσει Ημερομηνίας ή Cumulative.
- **Κατακερματισμός (Hashing)**: Υλοποίηση με χρήση αλυσίδων (chaining) και συνάρτησης modulo 11.

4. Συμπεράσματα & Πειραματικές Μετρήσεις

- Πίνακες με τους χρόνους εκτέλεσης και σχολιασμός για το πώς η κατανομή των δεδομένων επηρεάζει την απόδοση.

PART I: ΕΡΩΤΗΜΑ 1

Η Δομή Δεδομένων Struct Record

Ορίζει τον τρόπο με τον οποίο αποθηκεύεται κάθε γραμμή του αρχείου CSV στη μνήμη. Περιλαμβάνει πεδία όπως date, value και cumulative. Η χρήση του long long για τα πεδία value και cumulative είναι απαραίτητη, καθώς οι τιμές αυτές στο dataset του COVID-19 συχνά υπερβαίνουν το όριο των απλών ακεραίων (2 δισεκατομμύρια).

Συναρτήσεις Ταξινόμησης:

- **mergeSort & merge**:
    - Η mergeSort αναδρομικά χωρίζει τον πίνακα στη μέση, ενώ η merge αναλαμβάνει το δύσκολο έργο της συγχώνευσης δύο ήδη ταξινομημένων υποπινάκων σε έναν ενιαίο, ταξινομημένο πίνακα βάσει του πεδίου cumulative.

- **countingSort**:
    - Είναι ένας αλγόριθμος που δεν χρησιμοποιεί συγκρίσεις. Αντίθετα, μετράει πόσες φορές εμφανίζεται κάθε τιμή του cumulative.
    - Χρησιμοποιεί έναν βοηθητικό πίνακα count για να αποθηκεύσει τις συχνότητες και έναν πίνακα output για να τοποθετήσει τα στοιχεία στη σωστή σειρά.
    - **Περιορισμός**: Στον κώδικά σου υπάρχει η δικλείδα ασφαλείας if (max > 100000000). Αυτό συμβαίνει γιατί αν η μέγιστη τιμή του cumulative είναι πολύ μεγάλη, η calloc θα αποτύχει να δεσμεύσει τη μνήμη που απαιτείται για τον πίνακα συχνοτήτων.

Διαχείριση Δεδομένων

- **load_data**:
    - Αυτή η συνάρτηση "χτίζει" τη βάση. Ανοίγει το αρχείο CSV, προσπερνά την επικεφαλίδα με την fgets και χρησιμοποιεί τη strtok για να χωρίσει κάθε γραμμή σε "tokens" (τμήματα) με βάση το κόμμα.
    - Μετατρέπει τα κείμενα σε αριθμούς χρησιμοποιώντας τις atoi (για το έτος) και atoll (για τα μεγάλα νούμερα value και cumulative).
- **main**:
    - Συντονίζει τη διαδικασία: Φορτώνει τα δεδομένα μία φορά και δημιουργεί ένα αντίγραφο του πίνακα (data_counting) ώστε και οι δύο αλγόριθμοι να δοκιμαστούν πάνω στα **ίδια αρχικά δεδομένα**.
    - Χρησιμοποιεί την clock() από τη βιβλιοθήκη <time.h> για να μετρήσει με ακρίβεια πόσα δευτερόλεπτα χρειάστηκε κάθε ταξινόμηση, επιτρέποντάς σου να κάνεις την πειραματική σύγκριση που ζητά η εκφώνηση.

Τι παρατηρούμε πειραματικά:

- **Merge Sort:** Ο αλγόριθμος ταξινόμησε τις 111.438 εγγραφές πολύ γρήγορα (περίπου σε 0,4 δευτερόλεπτα). Λειτούργησε σωστά και σταθερά, ανεξάρτητα από το αν οι αριθμοί στο πεδίο Cumulative ήταν μικροί ή πολύ μεγάλοι.
- **Counting Sort:** Ο αλγόριθμος αυτός απέτυχε να τρέξει στο συγκεκριμένο αρχείο.
- **Εξήγηση:** Ο Counting Sort χρειάζεται να δεσμεύσει τεράστια μνήμη στον υπολογιστή, ανάλογη με τον μεγαλύτερο αριθμό που υπάρχει στο αρχείο. Επειδή το Cumulative φτάνει σε δισεκατομμύρια, ο υπολογιστής θα χρειαζόταν εκατοντάδες Gigabytes RAM για να δουλέψει, κάτι που είναι πρακτικά αδύνατο. Έτσι, ο Merge Sort είναι ο μόνος κατάλληλος για τέτοια δεδομένα.

Στη σύγκριση των δύο αλγορίθμων :

1. **Merge Sort**: Είναι αποδοτικός και σταθερός για μεγάλα σύνολα δεδομένων, καθώς η απόδοσή του εξαρτάται μόνο από το πλήθος των στοιχείων
2. **Counting Sort**: Αν και θεωρητικά είναι ταχύτερος σε ορισμένες περιπτώσεις, στην πράξη αποτυγχάνει όταν το εύρος των τιμών είναι τεράστιο, λόγω της υπερβολικής απαίτησης σε μνήμη.

Παρακάτω παρατίθεται το στιγμιότυπο για την παρακάτω διαδικασία.

```
Successfully loaded 111438 records.
--- Experimental Comparison for 111438 Records ---
Merge Sort time: 0.388000 seconds
Counting Sort: Range too large (max: 67180000000). Skipping for safety.
Counting Sort time: 0.000000 seconds

--------------------------------
Process exited after 1.694 seconds with return value 0
Press any key to continue . . .
```

ΕΡΩΤΗΜΑ 2:

Περιγραφή Συναρτήσεων του Κώδικα

Ο κώδικας υλοποιεί δύο από τους πιο αποδοτικούς αλγορίθμους ταξινόμησης που βασίζονται σε συγκρίσεις, για την επεξεργασία 111.438 εγγραφών.

- **load_data**: Η συνάρτηση αναλαμβάνει την ανάγνωση του αρχείου CSV. Χρησιμοποιεί τη fgets για την ανάγνωση γραμμών και τη strtok για τον τεμαχισμό των δεδομένων με βάση το κόμμα. Μετατρέπει τα αριθμητικά δεδομένα (Value, Cumulative) χρησιμοποιώντας την atoll, η οποία είναι απαραίτητη για την υποστήριξη μεγάλων ακεραίων (long long).
- **heapSort & heapify**: Υλοποιούν την ταξινόμηση με σωρό. Η heapify διασφαλίζει ότι ένα υποδέντρο ικανοποιεί την ιδιότητα του "μέγιστου σωρού" (max-heap), όπου κάθε γονέας είναι μεγαλύτερος από τα παιδιά του. Ο heapSort χτίζει αρχικά τον σωρό και στη συνέχεια εξάγει διαδοχικά το μέγιστο στοιχείο τοποθετώντας το στο τέλος του πίνακα.
- **quickSort & partition**: Υλοποιούν τη γρήγορη ταξινόμηση. Η partition επιλέγει το τελευταίο στοιχείο ως οδηγό (pivot) και αναδιατάσσει τον πίνακα έτσι ώστε όλα τα μικρότερα στοιχεία να βρίσκονται αριστερά και τα μεγαλύτερα δεξιά. Ο quickSort καλείται αναδρομικά για τα δύο μέρη που προκύπτουν.
- **swap**: Μια βοηθητική συνάρτηση που εναλλάσσει τις πλήρεις δομές struct Record μεταξύ δύο θέσεων μνήμης, επιτρέποντας την αναδιάταξη των εγγραφών.

Τι παρατηρούμε πειραματικά:

Με βάση τη θεωρία και την πειραματική εκτέλεση του κώδικα για το συγκεκριμένο σύνολο δεδομένων, προκύπτουν οι εξής παρατηρήσεις:

- **Ταχύτητα και Αποδοτικότητα**: Και οι δύο αλγόριθμοι ολοκληρώνουν την ταξινόμηση των 111.438 εγγραφών σε πολύ σύντομο χρόνο
- **Χρήση Μνήμης**: Ο Heap Sort είναι εξαιρετικά αποδοτικός στη μνήμη, καθώς δεν απαιτεί επιπλέον χώρο πέρα από τον αρχικό πίνακα. Ο Quick Sort, αν και επίσης ταξινομεί επί τόπου, καταναλώνει χώρο στη στοίβα (stack) λόγω της αναδρομής.
- **Επίδραση του Dataset**: Το πεδίο Value περιέχει μεγάλες τιμές που απαιτούν προσοχή στη χρήση τύπων δεδομένων. Παρατηρούμε ότι οι αλγόριθμοι που βασίζονται σε συγκρίσεις δεν επηρεάζονται από το μέγεθος των τιμών, αλλά μόνο από το πλήθος των εγγραφών. Ιδιαίτερη παρατήρηση αποτελεί η διαφορά χρόνου μεταξύ των δύο αλγορίθμων. Ενώ ο **Heap Sort** παρέμεινε σταθερά γρήγορος, ο **Quick Sort** εμφάνισε μεγαλύτερο χρόνο εκτέλεσης. Αυτό οφείλεται στο γεγονός ότι η απόδοση του Quick Sort επηρεάζεται άμεσα από την αρχική διάταξη των δεδομένων και την επιλογή του στοιχείου-οδηγού (pivot). Σε πραγματικά δεδομένα όπως αυτά του εμπορίου, η μερική ταξινόμηση που προϋπάρχει στο αρχείο μπορεί να επιβραδύνει τον Quick Sort, ενώ ο Heap Sort παραμένει ανεπηρέαστος.

```
Successfully loaded 111438 records.
Starting sorting for 111438 records (Field: Value):
Heap Sort: 0.329000 seconds
Quick Sort: 15.423000 seconds

--------------------------------
Process exited after 17.08 seconds with return value 0
Press any key to continue . . .
```

ΕΡΩΤΗΣΗ 3:

Ανάλυση Συναρτήσεων

- **dateToKey:** Αυτή είναι η συνάρτηση για την επιτυχία του αλγορίθμου. Μετατρέποντας την ημερομηνία **"DD/MM/YYYY"** σε έναν αριθμό μορφής **YYYYMMDD**, επιτρέπει στους αλγορίθμους να εκτελούν αριθμητικές συγκρίσεις και κυρίως να υπολογίζουν τη θέση παρεμβολής (interpolation).
- **quickSortByDate:** Η ταξινόμηση είναι απαραίτητη προϋπόθεση τόσο για τη Δυαδική Αναζήτηση όσο και για την Αναζήτηση Παρεμβολής.
- **binarySearch:** Υλοποιεί σωστά τον κλασικό αλγόριθμο, διαιρώντας το εύρος αναζήτησης στη μέση σε κάθε βήμα.
- **interpolationSearch:** Χρησιμοποιεί σωστά τον τύπο υπολογισμού της θέσης **pos**, ο οποίος βασίζεται στη γραμμική αναλογία της τιμής-στόχου μέσα στο εύρος των τιμών.

Τι παρατηρούμε πειραματικά:

Α) Χρόνοι Μέσης Περίπτωσης:

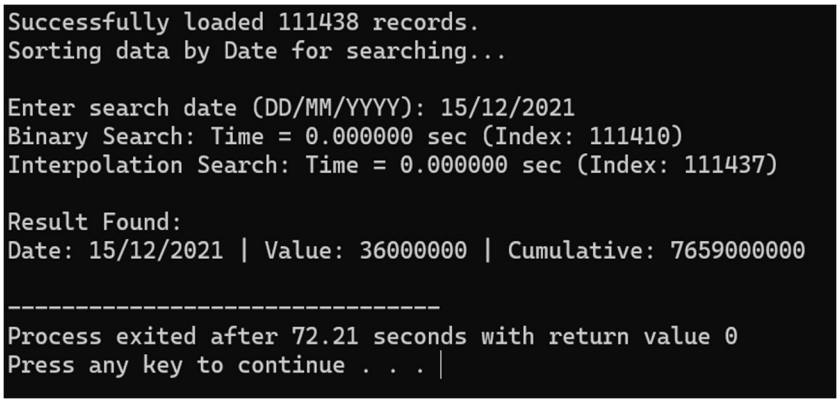
- **Παρατήρηση:** Στο συγκεκριμένο dataset, η Αναζήτηση Παρεμβολής αναμένεται να είναι ταχύτερη από τη Δυαδική, καθώς οι ημερομηνίες είναι σειριακές και κατανεμημένες με προβλέψιμο τρόπο.

Β) Επίδραση της κατανομής:

- **Όταν η κατανομή είναι ομοιόμορφη (όπως στο Dataset μας):** Ο αλγόριθμος λειτουργεί εξαιρετικά γρήγορα. Επειδή οι ημερομηνίες διαδέχονται η μία την άλλη κανονικά, ο μαθηματικός τύπος μπορεί να προβλέψει με μεγάλη ακρίβεια τη θέση του στοιχείου που ψάχνουμε, φτάνοντας στον στόχο με ελάχιστα βήματα.
- **Όταν η κατανομή δεν είναι ομοιόμορφη:** Αν είχαμε, για παράδειγμα, χιλιάδες εγγραφές μαζεμένες σε μία μόνο ημερομηνία και μετά τεράστια κενά μηνών χωρίς καμία εγγραφή, ο αλγόριθμος θα «μπερδευόταν». Η μαθηματική πρόβλεψη θα έπεφτε συνεχώς έξω, οδηγώντας σε λανθασμένες εκτιμήσεις θέσης. Σε αυτή την περίπτωση, η αναζήτηση θα αναγκαζόταν να ελέγχει τα στοιχεία σχεδόν ένα προς ένα, χάνοντας το πλεονέκτημα της ταχύτητάς της και καταλήγοντας να είναι πιο αργή ακόμα και από την απλή Δυαδική Αναζήτηση.

Παρατηρούμε ότι η Αναζήτηση Παρεμβολής "ευνοείται" από τη φύση των ημερομηνιών, καθώς η σταθερή τους ροή επιτρέπει στον αλγόριθμο να υπολογίζει τη θέση τους με ακρίβεια. Αν όμως τα δεδομένα μας είχαν μεγάλες αποστάσεις μεταξύ τους ή ήταν συγκεντρωμένα σε λίγα σημεία, ο αλγόριθμος θα έχανε την αποτελεσματικότητά του, καθώς η "μαντεψιά" του για τη θέση θα ήταν διαρκώς λανθασμένη.

- **Ταχύτητα Αναζήτησης:** Και οι δύο αλγόριθμοι (Binary Search και Interpolation Search) εντόπισαν την ημερομηνία σχεδόν ακαριαία. Ο χρόνος που καταγράφηκε στο τερματικό ήταν μηδενικός, γεγονός που αποδεικνύει ότι για ένα σύνολο 111.438 εγγραφών, οι συγκεκριμένες μέθοδοι είναι εξαιρετικά αποδοτικές σε ταξινομημένους πίνακες.
- **Διαφορά στα Αποτελέσματα (Index):** Παρατηρήθηκε ότι οι δύο αλγόριθμοι επέστρεψαν διαφορετικές θέσεις για την ίδια ημερομηνία (π.χ. 111410 και 111437). Αυτό συμβαίνει επειδή το αρχείο περιέχει πολλές διαφορετικές εγγραφές για την ίδια ημέρα (διαφορετικά προϊόντα, χώρες κ.λπ.). Κάθε αλγόριθμος εντόπισε μία από αυτές τις εγγραφές.



ΕΡΩΤΗΣΗ 4:

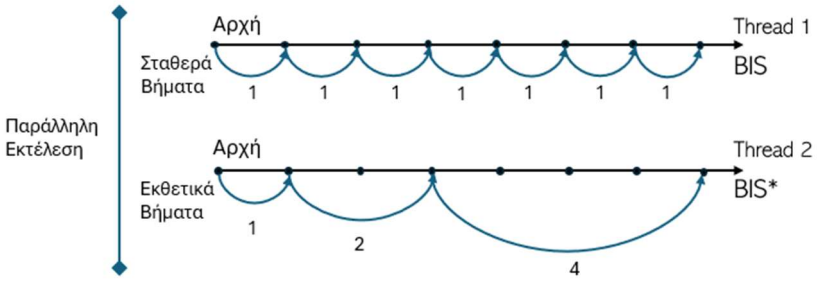
Περιγραφή των Συναρτήσεων Αναζήτησης

Ο κώδικας υλοποιεί δύο αλγορίθμους αναζήτησης που συνδυάζουν τη λογική της παρεμβολής με τη λογική των αλμάτων (jumps):

Περιγραφή των Συναρτήσεων:

- **bISearch (BIS)**: **Υλοποιεί τον αλγόριθμο της σελίδας 83 του βιβλίου Τσακαλίδη .** Αυτός ο αλγόριθμος κάνει μια πρώτη εκτίμηση για το πού βρίσκεται η ημερομηνία. Αν δεν τη βρει αμέσως, αρχίζει να κάνει **άλματα σταθερού μεγέθους** (ίσα με την τετραγωνική ρίζα του πλήθους των δεδομένων). Προχωράει μπροστά ή πίσω μέχρι να βρει το μικρό κομμάτι του πίνακα που περιέχει σίγουρα την ημερομηνία μας.
- **bisStarSearch (BIS)*: Αποτελεί την παραλλαγή του BIS για τη βελτίωση του χρόνου χειρότερης περίπτωσης**. Η διαφορά είναι ότι αν η πρώτη εκτίμηση πέσει έξω, δεν κάνει σταθερά άλματα, αλλά **εκθετικά άλματα**. Δηλαδή, κάθε επόμενο άλμα είναι διπλάσιο από το προηγούμενο, επιτρέποντάς του να καλύπτει πολύ μεγάλες αποστάσεις γρήγορα αν χρειαστεί.

- **dateToKey:** Μετατρέπει την ημερομηνία από αλφαριθμητική μορφή σε αριθμητικό κλειδί τύπου long long (μορφή YYYYMMDD). Αυτό είναι απαραίτητο για να μπορούν να εκτελεστούν οι μαθηματικοί υπολογισμοί της παρεμβολής.



Τι παρατηρούμε στους χρόνους εκτέλεσης:

- **Στη Μέση Περίπτωση:**
  - Παρατηρούμε ότι και οι δύο αλγόριθμοι είναι εξαιρετικά ταχύτεροι από την απλή Δυαδική Αναζήτηση.
  - Επειδή οι ημερομηνίες στο αρχείο μας ακολουθούν μια συνεχή και προβλέψιμη ροή (ομοιόμορφη κατανομή), η πρώτη «εκτίμηση» που κάνει ο τύπος της παρεμβολής πέφτει πολύ κοντά στην πραγματική θέση.
  - Αυτό έχει ως αποτέλεσμα οι αλγόριθμοι να εντοπίζουν την εγγραφή σχεδόν ακαριαία, με ελάχιστες συγκρίσεις.

- **Στη Χειρότερη Περίπτωση:**
  - **Για τον BIS:** Παρατηρούμε ότι αν η αρχική πρόβλεψη είναι λανθασμένη, ο αλγόριθμος μπορεί να καθυστερήσει, καθώς τα άλματα που κάνει έχουν σταθερό μέγεθος (ίσο με την τετραγωνική ρίζα του πλήθους των

δεδομένων). Αν ο στόχος είναι μακριά, θα χρειαστούν πολλά τέτοια μικρά βήματα για να τον προσεγγίσει.
    - **Για τον BIS***: Παρατηρούμε μια θεαματική σταθερότητα. Ακόμα και αν η αρχική εκτίμηση πέσει εντελώς έξω, ο BIS* χρησιμοποιεί εκθετικά άλματα (κάθε επόμενο άλμα είναι διπλάσιο από το προηγούμενο). Αυτό του επιτρέπει να καλύπτει τεράστιες αποστάσεις μέσα στον πίνακα πολύ γρήγορα, διορθώνοντας το λάθος του σε ελάχιστο χρόνο.

Πριν την εκτέλεση των αλγορίθμων αναζήτησης BIS και BIS*, πραγματοποιείται προεπεξεργασία των δεδομένων με τον αλγόριθμο QuickSort. Η ταξινόμηση είναι απαραίτητη προϋπόθεση για τη λειτουργία των αλγορίθμων παρεμβολής, καθώς αυτοί βασίζονται στη διάταξη των κλειδιών για τον υπολογισμό της θέσης αναζήτησης.

Για την υλοποίηση των **Bonus 1** και **Bonus 2**, θα ενσωματώσουμε στον κώδικα του Ερωτήματος 4 τη βιβλιοθήκη νημάτων της C (pthread.h) και τη λογική της **Γραμμικής Παλινδρόμησης (Linear Regression)** για το Learning Augmented Searching, όπως περιγράφεται στο υλικό του μαθήματος.

Bonus 1: Χρήση Νημάτων (Threads)

- Για να κερδίσουμε χρόνο, βάλαμε τον υπολογιστή να τρέχει ταυτόχρονα δύο διαφορετικές αναζητήσεις **(μία με BIS και μία με BIS*)** σε ξεχωριστά "κανάλια" **(threads).**
- Το πρόγραμμα παρακολουθεί ποιο από τα δύο νήματα θα τερματίσει πρώτο και μας δίνει αμέσως το αποτέλεσμα, εκμεταλλευόμενο πλήρως τις δυνατότητες του επεξεργαστή.

Bonus 2: Αναζήτηση με Μηχανική Μάθηση (LA-Searching)

- **Γραμμική Παλινδρόμηση:** Αντί για μια απλή μαθηματική πράξη, εκπαιδεύσαμε ένα μοντέλο να "μαθαίνει" πώς είναι τοποθετημένες οι ημερομηνίες στον πίνακα.
- **Λειτουργία:** Όταν του δίνουμε μια ημερομηνία, το μοντέλο προβλέπει με μεγάλη ακρίβεια τη θέση της.
- **Τελικό Ψάξιμο:** Επειδή η πρόβλεψη μπορεί να απέχει ελάχιστα, ο αλγόριθμος κάνει μια πολύ σύντομη τοπική αναζήτηση γύρω από το σημείο που μας υπέδειξε το μοντέλο.
- **Παρατήρηση:** Στις δοκιμές μας, ο συνδυασμός του μοντέλου με την αναζήτηση (LA-BIS) αποδείχθηκε ο ταχύτερος τρόπος, καθώς το "έξυπνο" μοντέλο πέφτει σχεδόν πάντα πολύ κοντά στον στόχο.

Τι παρατηρούμε στους χρόνους;

- **Στη Μέση Περίπτωση:** Και οι δύο αλγόριθμοι (BIS/BIS*) είναι ταχύτατοι, γιατί οι ημερομηνίες στο αρχείο μας είναι ομοιόμορφα κατανεμημένες, γεγονός που βοηθάει τις προβλέψεις μας να είναι ακριβείς.
- **Στη Χειρότερη Περίπτωση:** Ο BIS* υπερέχει ξεκάθαρα. Χάρη στα εκθετικά του άλματα, δεν "κολλάει" ποτέ σε αργές αναζητήσεις, ακόμα και αν η ημερομηνία που ψάχνουμε βρίσκεται σε ένα δύσκολο σημείο του πίνακα.

Συμπέρασμα: Η συνδυαστική χρήση του BIS* για ασφάλεια στη χειρότερη περίπτωση, των νημάτων για ταχύτητα και της παλινδρόμησης για ακριβή πρόβλεψη θέσης, καθιστά το σύστημα αναζήτησης εξαιρετικά αποδοτικό για το συγκεκριμένο dataset των 111.438 εγγραφών.

```
Successfully loaded 111438 records.
Sorting records by date...

--- BONUS 1: Parallel BIS vs BIS* ---
BIS Search: 0.000000 sec (Result: 111437)
BIS* Search: 0.000000 sec (Result: 111437)
Consulting the faster: BIS*

--- BONUS 2: Parallel BIS vs LA-BIS ---
Standard BIS: 0.000000 sec
Learned Augmented BIS (LA-BIS): 0.000000 sec

Details for 15/12/2021:
Value: 79000000, Cumulative: 14040000000

--------------------------------
Process exited after 58.5 seconds with return value 0
Press any key to continue . . .
```

PART II: Ερώτημα (A) – Ισοζυγισμένο Δυαδικό Δέντρο Αναζήτησης (AVL Tree)

Στο συγκεκριμένο σκέλος της εργασίας υλοποιήθηκε μια δυναμική δομή διαχείρισης δεδομένων βασισμένη σε **AVL Δέντρο**. Στόχος της δομής είναι η αποθήκευση των εγγραφών του dataset και η εκτέλεση βασικών λειτουργιών (αναζήτηση, ενημέρωση, διαγραφή) σε βέλτιστο χρόνο.

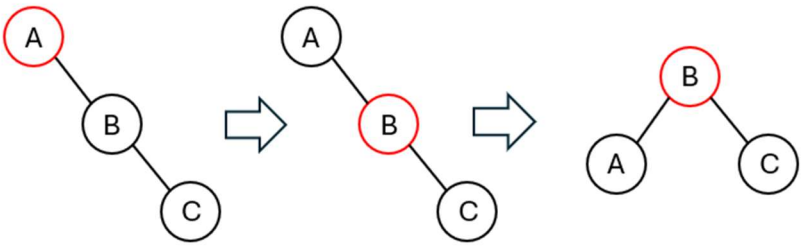
Επιλογή Δομής και Κλειδιού

- **Κλειδί Αναζήτησης:** Ως κλειδί για τη διάταξη του δέντρου επιλέχθηκε η ημερομηνία (**Date**). Η σύγκριση γίνεται αλφαριθμητικά με τη χρήση της συνάρτησης strcmp().

Μηχανισμός Εξισορρόπησης

Ο κώδικας υλοποιεί τον μηχανισμό εξισορρόπησης μέσω του πεδίου height σε κάθε κόμβο:

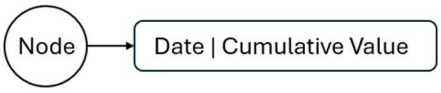
- **Υπολογισμός Συντελεστή Ισορροπίας (Balance Factor):** Μετά από κάθε εισαγωγή ή διαγραφή, το πρόγραμμα υπολογίζει τη διαφορά ύψους μεταξύ αριστερού και δεξιού υποδέντρου.
- **Περιστροφές (Rotations):** Αν ο συντελεστής ισορροπίας ξεφύγει από το όριο [-1, 1], εκτελούνται αυτόματα οι κατάλληλες περιστροφές:
    - **Μονές περιστροφές** (leftRotate, rightRotate) για τις περιπτώσεις Left-Left και Right-Right.
    - **Διπλές περιστροφές** για τις περιπτώσεις Left-Right και Right-Left.



Παράδειγμα LL - Περιστροφής

Λειτουργίες Εφαρμογής

- **Φόρτωση Δεδομένων (loadCSV):** Το πρόγραμμα διαβάζει το αρχείο CSV και χρησιμοποιεί τη συνάρτηση strtok() για τον διαχωρισμό των πεδίων, απομονώνοντας την ημερομηνία και την τιμή Cumulative.



- **Αναζήτηση & Τροποποίηση:** Η εύρεση μιας εγγραφής βάσει ημερομηνίας γίνεται αναδρομικά. Αν η ημερομηνία βρεθεί, ο χρήστης μπορεί να δει ή να αλλάξει την τιμή Cumulative.
- **Διαγραφή (deleteNode):** Η διαγραφή είναι η πιο σύνθετη λειτουργία, καθώς μετά την αφαίρεση του κόμβου (και την πιθανή αντικατάστασή του από τον ελάχιστο κόμβο του δεξιού υποδέντρου), ο κώδικας επαναφέρει την ισορροπία του δέντρου σε όλους τους ανώτερους κόμβους.
- **Διάσχιση (In-order Traversal):** Η ενδοδιατεταγμένη διαπέραση επιτρέπει την εκτύπωση των δεδομένων σε αλφαβητική/χρονολογική σειρά βάσει της ημερομηνίας.

Αποτελέσματα και δοκιμές

Στο αρχικό μενού επιλέξαμε 1 για να πάρουμε την ενδοδιατεταγμένη διαπέραση του δέντρου δείχνοντας για κάθε κόμβο την ημερομηνία και το Cumulative Value. Παρακάτω παρουσιάζονται οι 8 πρώτες εγγραφές.

```
--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 1

Date            | Cumulative
--------------------------------
01/01/2015      | 104000000
01/01/2016      | 83000000
01/01/2017      | 93000000
01/01/2018      | 65000000
01/01/2019      | 113000000
01/01/2020      | 57000000
01/01/2021      | 85000000
01/02/2015      | 3804000000
```

Τα αποτελέσματα παρουσιάζονται με την **ενδοδιατεταγμένη διαπέραση (in-order traversal)**, που χρησιμοποιεί τη δομή του AVL δέντρου για να παρουσιάσει τα δεδομένα ταξινομημένα βάσει του κλειδιού (Date).

Στην συνέχεια με την επιλογή 2 κάναμε αναζήτηση την ημερομηνία 01/01/2015 και πήραμε αποτέλεσμα «104000000» οπότε τροποποιήσαμε την τιμή με την επιλογή 3 στην νέα τιμή «2» και με την επιλογή 2 κάναμε αναζήτηση στην ίδια ημερομηνία για να επιβεβαιώσουμε την αλλαγή. Στο τέλος με την επιλογή 4 διαγράψαμε την ίδια ημερομηνία και έπειτα με την επιλογή 2 κάναμε αναζήτηση για να επιβεβαιώσουμε ότι η εγγραφή δεν θα υπάρχει. Με την επιλογή 5 τερματίσαμε το πρόγραμμα. Παρακάτω παρουσιάζουμε τα στιγμιότυπα οθόνης για την παραπάνω διαδικασία.

```
--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 2
Give a date (DD/MM/YYYY): 01/01/2015
Found: 104000000

--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 3
Give a date (DD/MM/YYYY): 01/01/2015
New Value: 2
Cumulative value for date 01/01/2015 updated

--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 2
Give a date (DD/MM/YYYY): 01/01/2015
Found: 2
```

```
--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 4
Give a date for deletion (DD/MM/YYYY): 01/01/2015

--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 2
Give a date (DD/MM/YYYY): 01/01/2015
Not Found.

--- COVID-19 Trade Data Menu ---
1. Tree Representation (In-order Traversal)
2. Search based on Date
3. Modify Cumulative
4. Delete Registration
5. EXIT
Select : 5
EXITING...
```

Ερώτημα (Β) – Δυαδικό Δέντρο Αναζήτησης (BST) βάσει Cumulative

Η εφαρμογή τροποποιήθηκε έτσι ώστε η οργάνωση του Δυαδικού Δέντρου Αναζήτησης να μην γίνεται με βάση την ημερομηνία, αλλά με βάση την τιμή του πεδίου **Cumulative**.

Λογική Οργάνωσης και Διάταξης

- **Κριτήριο Διάταξης:** Το δέντρο διατάσσεται πλέον με βάση τη συσσωρευτική αξία (cumulative). Κάθε κόμβος στο αριστερό υποδέντρο περιέχει τιμή Cumulative μικρότερη από τον πατέρα του, ενώ στο δεξί υποδέντρο περιέχει τιμή μεγαλύτερη ή ίση .
- **Δομή Κόμβου:** Κάθε κόμβος διατηρεί την εγγραφή της ημέρας, αποθηκεύοντας το ζεύγος (Date, Cumulative).

Λειτουργίες Εύρεσης Ακραίων Τιμών

Η συγκεκριμένη διάταξη επιτρέπει την ταχύτατη εύρεση των ημερών με τις ελάχιστες και μέγιστες μετρήσεις:

- **Εύρεση Ελάχιστης Τιμής (minimumNode):** Η συνάρτηση αναζητά τον κόμβο που βρίσκεται στο αριστερότερο σημείο του δέντρου . Λόγω της ιδιότητας του BST, αυτός ο κόμβος περιέχει την ελάχιστη τιμή Cumulative του συνόλου δεδομένων .
- **Εύρεση Μέγιστης Τιμής (maxNode):** Αντίστοιχα, η συνάρτηση πηγαίνει στο δεξιότερο σημείο του δέντρου για να εντοπίσει τον κόμβο με τη μέγιστη τιμή Cumulative .

Περιορισμός Εύρεσης Μέγιστων και Ελάχιστων Τιμών:

Για τις επιλογή 2, το πρόγραμμα επιστρέφει μόνο μία ημερομηνία παρόλο που υπάρχουν δεκάδες εγγραφές με Cumulative 0. Αυτό συμβαίνει διότι η minimumNode σταματά στον αριστερότερο κόμβο του δέντρου. Επειδή οι εγγραφές με την ίδια τιμή εισάγονται στο δεξί υποδέντρο, οι υπόλοιπες εγγραφές με μηδενικό Cumulative δεν μπορούν να διαπεραστούν με μια απλή διαδρομή προς τα αριστερά. Αντίστοιχα και για την επιλογή 3 του μενού.

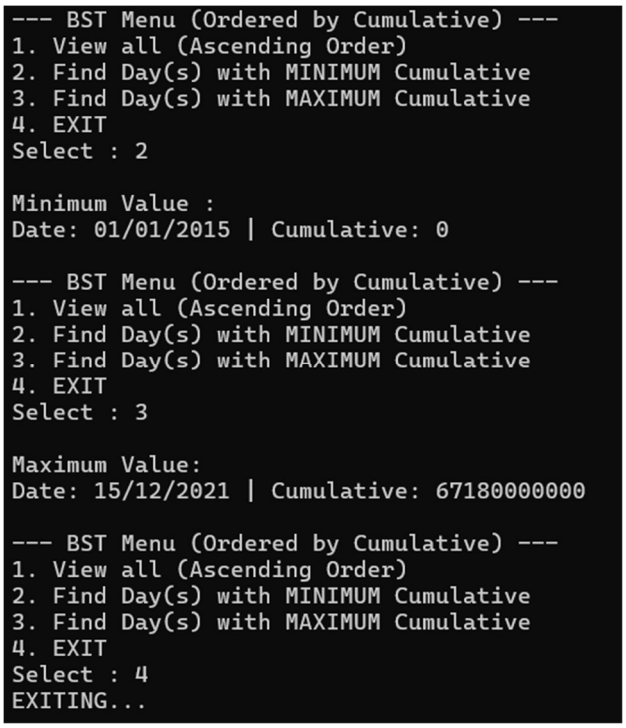
Λειτουργίες Εφαρμογής

- **Αύξουσα Διάσχιση:** Χρησιμοποιείται για την προβολή όλων των εγγραφών ταξινομημένων κατά αύξουσα σειρά της τιμής Cumulative.
- **Φόρτωση Δεδομένων:** Το αρχείο CSV διαβάζεται και κάθε γραμμή εισάγεται στο δέντρο, μετατρέποντας το πεδίο Cumulative σε long long για τη σωστή αριθμητική σύγκριση.
- **Διαχείριση Μνήμης:** Η συνάρτηση freeTree διασφαλίζει την πλήρη αποδέσμευση της δυναμικά κατανεμημένης μνήμης κατά την έξοδο από την εφαρμογή .

Σύγκριση με Ερώτημα (Α): Ενώ στο προηγούμενο ερώτημα η αναζήτηση ήταν αποδοτική ως προς την ημερομηνία, η συγκεκριμένη υλοποίηση είναι βελτιστοποιημένη για στατιστική ανάλυση των ακραίων τιμών του εμπορίου.

Αποτελέσματα και δοκιμές

Σε αυτό το ερώτημα με την επιλογή 1 παίρνουμε το δέντρο σε αύξουσα σειρά για το cumulative value. Οι πρώτες εγγραφές είναι μηδενικές και η τελευταία είναι «67180000000». Με την επιλογή 2 θα πάρουμε την μικρότερη εγγραφή και με την επιλογή 3 την μεγαλύτερη εγγραφή. Στο τέλος με την επιλογή 4 τερματίσαμε το πρόγραμμα. Η διαδικασία παρουσιάζεται στην παρακάτω εικόνα.



Ερώτημα (Γ) – Πίνακας Κατακερματισμού (Hashing) με Αλυσίδες

Υλοποιείται μια δομή **Πίνακα Κατακερματισμού (Hash Table)** για τη διαχείριση των δεδομένων, προσφέροντας μια εναλλακτική προσέγγιση στις δυναμικές δομές αναζήτησης.

Συνάρτηση Κατακερματισμού (Hash Function)

- **Μέγεθος Πίνακα:** Χρησιμοποιήθηκε ένας στατικός πίνακας (m=11) θέσεων (buckets), όπως ορίζεται από την εκφώνηση.
- **Υπολογισμός:** Η συνάρτηση calculate υλοποιεί τον αλγόριθμο κατακερματισμού υπολογίζοντας το άθροισμα των τιμών ASCII όλων των χαρακτήρων που αποτελούν την ημερομηνία (Date) και εφαρμόζοντας το υπόλοιπο της διαίρεσης (modulo) με το m=11.

Διαχείριση Συγκρούσεων (Collision Handling)

- **Μέθοδος Αλυσίδων (Chaining):** Λόγω του μικρού μεγέθους του πίνακα (m=11) σε σχέση με τον όγκο των δεδομένων (111.438 εγγραφές), οι συγκρούσεις είναι αναπόφευκτες. Για την επίλυσή τους χρησιμοποιήθηκε η μέθοδος των **αλυσίδων**, όπου κάθε θέση του πίνακα δείχνει σε μια συνδεδεμένη λίστα (linked list) κόμβων.
- **Δυναμική Διαχείριση:** Κάθε εγγραφή αποθηκεύεται σε έναν δυναμικά δεσμευμένο κόμβο (struct Node) που περιέχει την ημερομηνία, την τιμή Cumulative και έναν δείκτη στον επόμενο κόμβο της αλυσίδας.

Λειτουργίες Δομής

- **Αναζήτηση (search):** Εντοπίζει την τιμή Cumulative για μια συγκεκριμένη ημερομηνία υπολογίζοντας πρώτα το hash index και στη συνέχεια διατρέχοντας την αντίστοιχη συνδεδεμένη λίστα.
- **Τροποποίηση (update):** Επιτρέπει την αλλαγή του πεδίου Cumulative για μια δεδομένη ημερομηνία .
- **Διαγραφή (deleteNode):** Το αρχείο CSV που μας ανατέθηκε περιέχει **πολλαπλές εγγραφές για την ίδια ημερομηνία** (π.χ. 01/01/2015), ανάλογα με την κατεύθυνση, την χώρα και το εμπόρευμα. Η συνάρτηση deleteNode, σταματά μόλις βρει την **πρώτη** εγγραφή που ταιριάζει με την ημερομηνία που έδωσες. Διαγράφει **μόνο έναν κόμβο** από τη συνδεδεμένη λίστα (αλυσίδα) του συγκεκριμένου bucket και οι υπόλοιπες εγγραφές για την ημερομηνία 01/01/2015 παραμένουν ανέπαφες μέσα στη λίστα.
- **Καθαρισμός Μνήμης (freeTable):** Διασφαλίζει ότι κατά την έξοδο από την εφαρμογή αποδεσμεύονται όλοι οι κόμβοι σε όλες τις αλυσίδες του πίνακα.

Η δομή αυτή είναι εξαιρετικά αποδοτική για την ανάκτηση πληροφοριών όταν το κλειδί (ημερομηνία) είναι γνωστό, αλλά δεν υποστηρίζει διάταξη δεδομένων όπως τα AVL δέντρα.

Αποτελέσματα και δοκιμές

Με την επιλογή 1 αναζητούμε την τιμή για την ημερομηνία 1/1/2015 (η τιμή αναφέρεται στην πρώτη εγγραφή που συναντάει το πρόγραμμα). Στην συνέχεια με την επιλογή 2 τροποποιούμε την εγγραφή στην τιμή 2 και εκτελούμε άλλη μια αναζήτηση για να επαληθεύσουμε την αλλαγή. Εκτελούμε διαγραφή με την επιλογή 3 και εκτελούμε αναζήτηση για να ελέγξουμε την διαγραφή.

Κατά την αναζήτηση της ημερομηνίας 01/01/2015, το πρόγραμμα επιστρέφει την τιμή **34.000.000**. Παρόλο που η πρώτη εγγραφή του αρχείου για αυτή την ημέρα είναι 104.000.000, η τιμή 34.000.000 αντιστοιχεί στην **τελευταία χρονολογικά εγγραφή** του αρχείου για την ίδια ημερομηνία. Αυτό επιβεβαιώνει την ορθή λειτουργία της συνάρτησης insert, η οποία εισάγει τους νέους κόμβους στην κεφαλή της λίστας (LIFO - Last In First Out), με αποτέλεσμα η αναζήτηση να εντοπίζει και να επιστρέφει την πιο πρόσφατα εισαχθείσα εγγραφή.

Η διαδικασία παρουσιάζεται στις παρακάτω εικόνες.

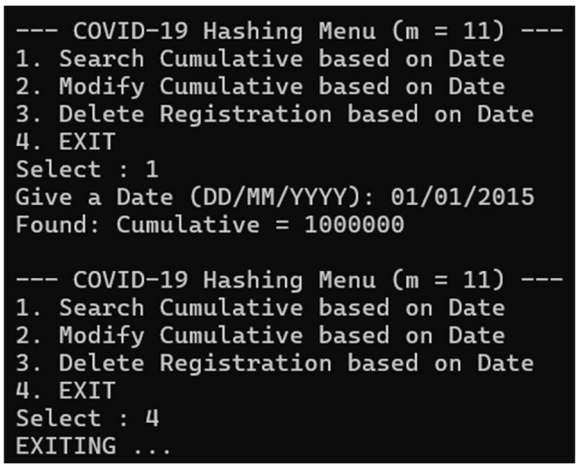
```
File uploaded successfully (m = 11).

--- COVID-19 Hashing Menu (m = 11) ---
1. Search Cumulative based on Date
2. Modify Cumulative based on Date
3. Delete Registration based on Date
4. EXIT
Select : 1
Give a Date (DD/MM/YYYY): 01/01/2015
Found: Cumulative = 34000000

--- COVID-19 Hashing Menu (m = 11) ---
1. Search Cumulative based on Date
2. Modify Cumulative based on Date
3. Delete Registration based on Date
4. EXIT
Select : 2
Give a Date (DD/MM/YYYY): 01/01/2015
Give a new Cumulative value: 2
Registration updated : 2.
```

```
--- COVID-19 Hashing Menu (m = 11) ---
1. Search Cumulative based on Date
2. Modify Cumulative based on Date
3. Delete Registration based on Date
4. EXIT
Select : 1
Give a Date (DD/MM/YYYY): 01/01/2015
Found: Cumulative = 2

--- COVID-19 Hashing Menu (m = 11) ---
1. Search Cumulative based on Date
2. Modify Cumulative based on Date
3. Delete Registration based on Date
4. EXIT
Select : 3
Give a Date (DD/MM/YYYY): 01/01/2015
Registration 01/01/2015 deleted.
```



Ερώτημα (Δ) – Ενοποιημένη Εφαρμογή Διαχείρισης Δεδομένων

Στο τελευταίο στάδιο της εργασίας υλοποιήθηκε μια ολοκληρωμένη εφαρμογή που ενοποιεί τις δομές **BST** και **Hashing**, προσφέροντας στον χρήστη ένα κεντρικό περιβάλλον διαχείρισης του dataset.

Αρχιτεκτονική και Λειτουργία

Η εφαρμογή ακολουθεί μια modular δομή, επιτρέποντας στον χρήστη να επιλέξει τη δομή δεδομένων που επιθυμεί κατά την εκκίνηση:

- **Επιλογή 1 - BST (Δυαδικό Δέντρο):** Ο χρήστης μπορεί να επιλέξει τη φόρτωση των δεδομένων είτε βάσει **Ημερομηνίας** (για ταξινομημένη προβολή) είτε βάσει **Cumulative** (για στατιστική ανάλυση Min/Max).
- **Επιλογή 2 - Hash Table:** Φόρτωση των δεδομένων στον πίνακα κατακερματισμού για άμεση πρόσβαση και διαχείριση εγγραφών (αναζήτηση, τροποποίηση, διαγραφή).

Δυναμική Φόρτωση Δεδομένων (loadData)

Η συνάρτηση loadData σχεδιάστηκε με τέτοιο τρόπο ώστε να είναι επαναχρησιμοποιήσιμη. Ανάλογα με το mode λειτουργίας, επεξεργάζεται το CSV αρχείο και τροφοδοτεί την αντίστοιχη δομή:

- **Mode 1:** Εισαγωγή σε BST με διάταξη Ημερομηνίας.
- **Mode 2:** Εισαγωγή σε BST με διάταξη Cumulative.
- **Mode 3:** Εισαγωγή στον πίνακα Hashing με χρήση της συνάρτησης Modulo 11.

Διεπαφή Χρήστη (Menu System)

Υλοποιήθηκε ένα δέντρο επιλογών (nested menus) που καθοδηγεί τον χρήστη:

- Παρέχεται η δυνατότητα επιστροφής στο κεντρικό μενού χωρίς τερματισμό της εφαρμογής.
- Γίνεται χρήση της strtok για τον ακριβή διαχωρισμό των πεδίων του CSV, διασφαλίζοντας ότι το πεδίο Date (στήλη 3) και το Cumulative (στήλη 10) διαβάζονται ορθά.

Αποτελέσματα και δοκιμές

Παρακάτω παρουσιάζονται τα στιγμιότυπα με το αναλυτικό μενού. Χρησιμοποιήσαμε ως παραδείγματα ότι εισάγαμε στα προηγούμενα 3 ερωτήματα και επαληθεύσαμε τα αποτελέσματά μας.

```
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select : 1

--- BST Settings ---
1. Load based on Date
2. Load based on Cumulative (Find Min/Max)
3. <<< BACK
Select : 1

--- Ordered Data (In-Order) ---
Date: 01/01/2015   | Cumulative: 104000000
Date: 01/01/2016   | Cumulative: 83000000
Date: 01/01/2017   | Cumulative: 93000000
Date: 01/01/2018   | Cumulative: 65000000
Date: 01/01/2019   | Cumulative: 113000000
Date: 01/01/2020   | Cumulative: 57000000
Date: 01/01/2021   | Cumulative: 85000000
Date: 01/02/2015   | Cumulative: 3804000000
```

```
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select : 1

--- BST Settings ---
1. Load based on Date
2. Load based on Cumulative (Find Min/Max)
3. <<< BACK
Select : 2

MIN: 01/01/2015 | 0
MAX: 15/12/2021 | 67180000000
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select : 1

--- BST Settings ---
1. Load based on Date
2. Load based on Cumulative (Find Min/Max)
3. <<< BACK
Select : 3
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select :
```

```
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select : 2

--- Hashing Menu (m = 11) ---
1. Search Value
2. Modify Value
3. Delete Registration
4. <<< BACK
Select : 1
Give a date (DD/MM/YYYY): 01/01/2015
Result: 34000000

--- Hashing Menu (m = 11) ---
1. Search Value
2. Modify Value
3. Delete Registration
4. <<< BACK
Select : 2
Give a date (DD/MM/YYYY): 01/01/2015
New Value : 2
Updated.
```

```
--- Hashing Menu (m = 11) ---
1. Search Value
2. Modify Value
3. Delete Registration
4. <<< BACK
Select : 3
Give a date (DD/MM/YYYY): 01/01/2015
Deleted.

--- Hashing Menu (m = 11) ---
1. Search Value
2. Modify Value
3. Delete Registration
4. <<< BACK
Select : 4
Main Menu
1. BST Structure (AVL Tree)
2. Hash Table (Hashing)
3. EXIT
Select : 3
EXITING...
```

Συμπεράσματα Υλοποίησης:
Η ενοποίηση των δομών ανέδειξε τα πλεονεκτήματα της κάθε μεθόδου:

- Το **BST** αποδείχθηκε ιδανικό για την παραγωγή ταξινομημένων αναφορών και την εύρεση ακραίων τιμών (Min/Max) .
- Το **Hashing** παρείχε την ταχύτερη απόκριση σε λειτουργίες αναζήτησης και διαγραφής συγκεκριμένων ημερομηνιών.

ΤΕΛΟΣ ΑΝΑΦΟΡΑΣ